

HW2 無人機作業報告

Task B：隨機目標導航與 PPO 強化學習

姓名：蔡岳哲

學號：M143040001

大綱

一、任務定義與研究動機

二、現有方法的問題與限制

三、文獻回顧

四、提出的解決方案

五、實驗結果與討論

附錄 A、Part 1 P Controller 心得

附錄 B、訓練期不同 level 結果截圖

附錄 C、AI 工具輔助

一、任務定義與研究動機

1.1 任務定義

我選擇的 Task B 的目標是讓無人機學會前往「每回合隨機生成」的三維目標座標。

因此，實驗設計為：每個 episode 開始時，環境會在 level 定義的球形範圍內隨機選出一新目標，無人機必須飛過去、進入到抵達圈內（半徑 `ARRIVE_DIST`）之後，還要在速度低於 `HOVER_MAX` 的條件下連續懸停 `HOVER_STEPS` 步才算成功。

成功的評估用的是 AirPilot 論文裡提到的 `EffectiveSpeed`：

$$\text{EffectiveSpeed} = \text{初始距離} / \text{花費步數} \quad (\text{越高越好})$$

這個指標同時懲罰「飛太慢」、「衝過頭」和「懸停太久才穩定」。

1.2 為什麼值得研究

- 無人機模型沒辦法靠記路過關，因為每次目標都不一樣，所以需要真的要學會三維空間的導航邏輯，才能完成任務。
- 要同時做到快速抵達、穩定懸停，但飛得快衝過頭就算失敗，這會導致其比純粹的定點控制難的多。
- 7 個難度層級從簡單到難逐步增加，讓訓練過程更穩定，不會一開始就因為稀疏獎勵而訓不動。

二、現有方法的問題與限制

2.1 P 控制器的問題

單一增益無法同時應付遠近兩種情況

P 控制器只有一個增益 K_p ，對所有距離的誤差都用同一套邏輯處理。但我設計的任務從 L1 的 1 公尺到 L7 的 20 公尺以上都有，這就造成一個很難解的矛盾：

- K_p 調大：長距離飛得快沒問題，但接近目標時累積的動量太大，會直接衝過 0.6m 的抵達範圍圈，然後在兩側來回震盪。
- K_p 調小：近距離懸停比較穩，但長途飛行就慢到一個極致，而且距目標幾公尺的地方就已經動不了了，卡在穩態誤差出不去。

兩邊都無法完美達成，所以沒有一個 K_p 值能同時搞定。

沒有預測性煞車能力

P 控制器只看「現在的誤差有多大」，不會管「飛過來的速度多快」。在 L7 (大概 15 公尺) 的距離全速衝過來，進到 0.6m 的抵達範圍時速度可能還有 0.8~1.0 m/s，遠超過 0.2 m/s 的懸停門檻。接下來就是在目標點附近大幅擺動直到時間耗盡，觸發超時失敗。

2.2 PID 的通用限制

- 線性回饋的本質讓它很難整合多個訊號來源（位置、速度、傾角），通常需要額外設計外迴路才行。
- 本實驗的設計有 7 個難度，如果像傳統 PID 那樣每個難度都要人工重新調整，會造成運作上的麻煩。
- 沒辦法自己學會任務特有的策略，例如：
為了解決 20 公尺長途飛行的問題，將 K_p 調高以維持足夠的巡航速度，並將 K_d 調到極大，希望它能在接近目標時產生強大的虛擬煞車力，避免超過 0.6m 的範圍。
這個參數在 20 公尺的任務中或許勉強能煞住。但當放到 1 公尺的短距離任務時，因為 K_d 極高，飛機對任何微小的速度變化都會極度敏感。在僅有 1 公尺的移動過程中，感測器的微小雜訊會被 K_d 瘋狂放大，導致馬達產生高頻的震盪。。

三、文獻回顧

3.1 Tan & Karaköse (2023) — 分散式 PPO 無人機追蹤

引用： Tan, Z., & Karaköse, M. (2023). A New Approach for Drone Tracking with Drone Using Proximal Policy Optimization Based Distributed Deep Reinforcement Learning. *SoftwareX*, 23, 101497.

- 方法：用 PPO 做分散式深度強化學習，讓一架無人機自主追蹤另一架，並比較分散式與非分散式訓練的效能差異。
- 主要發現：PPO 在三維連續控制中具備良好的穩定性，論文也直接給出了在實驗中驗證過的最佳超參數。

對本實驗的影響：直接採用論文中的 [256, 256] 雙隱藏層 + Tanh 啟動函數的架構作為 policy network 和 value network 的設計基礎。

3.2 Zhang et al. (2025) — AirPilot PPO 自動調整非線性 PID

引用： Zhang, J.; Rivera, C.E.O.; Tyni, K.; Nguyen, S. AirPilot: Interpretable PPO-Based DRL Auto-Tuned Nonlinear PID Drone Controller for Robust Autonomous Flights. *arXiv:2404.00204v5*, 2025.

- 方法：PPO 不直接控馬達，而是即時調整 PID 控制器的參數。獎勵函數設計上以最大化 EffectiveSpeed 為目標，同時對過大的傾角給予重罰。
- 主要發現：把時間、距離、穩定性整合進單一獎勵指標，效果比只看步數好很多；傾角懲罰能有效避免危險的飛行姿態。

對本實驗的影響：借用了 EffectiveSpeed 作為主要評估指標，也參考了他們的傾角懲罰設計（軟限制 30°、硬限制 60° 墜機），以及靠近目標時的減速帶獎勵機制。

3.3 Shen & Huang (2024) — RL 應用於四旋翼飛行動作控制

引用： Shen, S.-E., & Huang, Y.-C. (2024). Application of Reinforcement Learning in Controlling Quadrotor UAV Flight Actions. *Drones*, 8(11), 660.

- 方法：比較 DQN、A2C、PPO 三種演算法讓無人機連續穿越 11 個目標框。提出 CRM（連續回合法）獎勵機制，加入角度精準度評分。
- 主要發現：PPO 收斂最快、平均獎勵最高；A2C 根本學不好連續導航任務。

對本實驗的影響：決定選用 PPO，並直接採用論文驗證過的超參數組合（lr=3e-4, n_steps=2048, batch_size=64, gae_lambda=0.95）距離差獎勵和每步時間懲罰也參考了他們的設計。

四、提出的解決方案

4.1 演算法選擇：PPO

- 選 PPO 主要基於三個原因：
- 透過 clipped surrogate objective 讓訓練更新很穩定。
 - 樣本效率夠高，適合跑 ROS 2 / Gazebo 這種每步都要實際模擬的環境。
 - 上面三篇文獻都用 PPO 並且效果很好。

4.2 MDP 定義

狀態空間（13 維）

項目	維度	說明	正規化方式
無人機絕對位置	3	x, y, z 座標	÷ 邊界值
目標絕對位置	3	目標 x, y, z	÷ 邊界值
相對位置向量	3	目標 - 無人機	÷ 邊界值
線性速度	3	vx, vy, vz	÷ MAX_SPEED
純量距離	1	目標 - 無人機	÷ 最大距離

動作空間（3 維）

輸出連續的三軸速度指令 $[vx, vy, vz] \in [-1, 1]$ m/s，正則化後以 10 Hz（每步 0.1 秒）發送到 /simple_drone/cmd_vel。

折扣因子

$\gamma = 0.99$ ，讓模型比較在意長期回報，才會學到「前期快速加速、後期主動煞車」的策略，而不是只顧眼前。

4.3 神經網路架構與超參數

參數	設定值	參考來源
網路架構	MLP [256, 256]，Tanh activation	Tan (2023)
Learning Rate	3×10^{-4}	Shen (2024)
n_steps	2048	Shen (2024)
batch_size	64	Shen (2024)
gae_lambda	0.95	Shen (2024)
vf_coef	0.5	Shen (2024)
gamma	0.99	Zhang (2025)

4.4 獎勵函數設計

每步的總獎勵 = 固定時間懲罰 + 以下 8 個細項組合：

項目	觸發條件 / 計算方式	數值
時間懲罰	每一步固定扣分	-0.2
進度獎勵（靠近）	與上步相比距離縮短	+25 × 距離差

項目	觸發條件 / 計算方式	數值
進度懲罰（遠離）	與上步相比距離增加	$+35 \times \text{距離差（負）}$
速度方向對齊	在抵達範圍圈外且有速度	$\cos(\text{速度向量, 目標方向}) \times \text{權重}$
減速帶懲罰	距目標 $< 2 \times \text{ARRIVE_DIST}$ 且速度 $> \text{HOVER_MAX}$	依距離比例的漸進懲罰
傾角懲罰	傾角 $> 30^\circ$ （軟）/ $> 60^\circ$ （墜機）	線性懲罰 / 回合終止
動作 L2 正則化	每步	$-\lambda \times \ \text{動作}\ ^2$
接近獎勵	在抵達範圍圈內且速度符合	越近、越慢，分數越高
到達獎勵	連續懸停達 HOVER_STEPS 步	高額獎勵 + 品質加成， 回合終止
墜毀懲罰	出界或傾角 $> 60^\circ$	-100 ， 回合終止

4.5 漸進式課程學習

如果一開始就在 L7 這麼大的範圍訓練的話，目標會太難找、而且獎勵太稀疏，訓練會訓不動。

所以把難度拆成 7 個階段慢慢升：

- 每 50 個回合跑一次確定性評估（20 回合）。
- 成功率達到 65% 就自動升難度。
- 難度提升的方向：目標生成範圍擴大、抵達範圍圈縮小、懸停速度門檻降低、需要懸停的步數增加。

4.6 課程難度設計（L1 - L7）

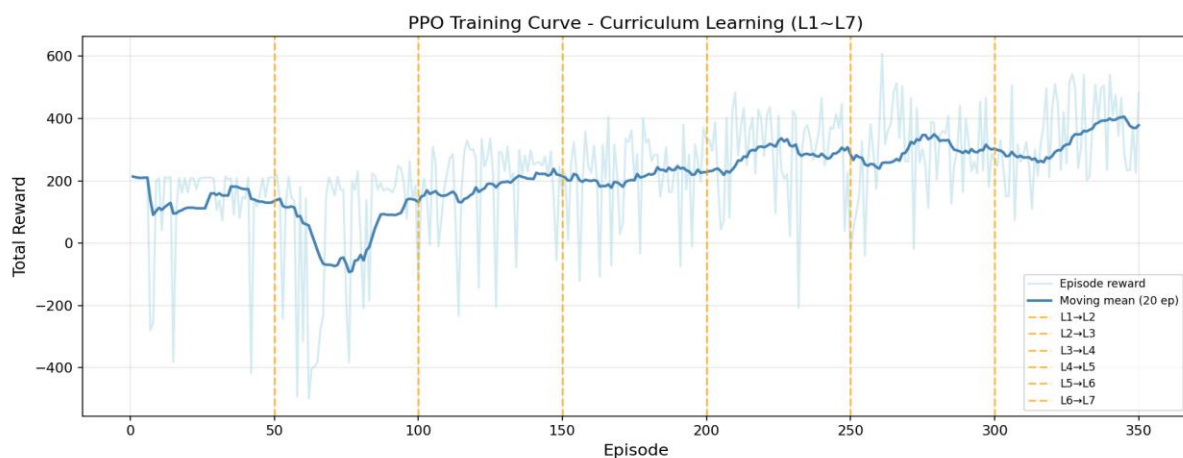
難度	目標生成半徑（m）	抵達範圍圈半徑（m）	懸停速度上限（m/s）	懸停步數
L1	1.5	1.00	0.40	5
L2	3.0	0.90	0.35	8
L3	5.0	0.80	0.30	10
L4	7.0	0.75	0.25	15
L5	8.5	0.70	0.22	18
L6	10.0	0.65	0.20	20
L7	11.0	0.60	0.20	20

五、實驗結果與討論

5.1 訓練曲線分析

整個訓練過程大概可以分三個階段：

- L1 - L2（快速收斂）：目標生成範圍小、到達條件寬鬆，無人機可以在不到 10 步就完成任務，獎勵曲線幾乎沒什麼降低，或是直接爬上去的。
- L3 - L6（過渡期震盪）：距離拉長後，每步固定的時間懲罰在長途飛行中累積很快，無人機需要從之前全速衝撞的策略轉型，學習如何在接近目標前主動煞車。這段大概每個難度會需要 100~200 回合才能重新收斂。
- L7（最終收斂）：確定性評估成功率穩定在 80%，說明模型已經學會預測性煞車了。



圖一：訓練獎勵曲線 (L1 → L7)

```
[L1] Ep 10 | Mean: 113.58 | Prog: -11.46 | Align: -1.95 | Prox: 6.19 | Decel: 0.00 | Tilt: 0.00 | Acti: -1.94 | Arrive: 157.69 | Time: -34.96 | Bound: 0.00
-----
rollout/
  ep_len_mean 110
  ep_rev_mean 129
time/
  fps 7
  iterations 1
  time_elapsed 264
  total_timesteps 2048
-----
[L1] Ep 20 | Mean: 114.60 | Prog: -6.98 | Align: 1.64 | Prox: 5.53 | Decel: 0.00 | Tilt: 0.00 | Acti: -1.64 | Arrive: 147.33 | Time: -21.28 | Bound: -10.00
[L1] Ep 30 | Mean: 195.61 | Prog: 3.18 | Align: 1.07 | Prox: 6.73 | Decel: 0.00 | Tilt: 0.00 | Acti: -0.20 | Arrive: 187.53 | Time: -2.70 | Bound: 0.00
[L1] Ep 40 | Mean: 151.27 | Prog: 4.40 | Align: 5.13 | Prox: 6.11 | Decel: 0.00 | Tilt: 0.00 | Acti: -0.99 | Arrive: 148.94 | Time: -12.32 | Bound: 0.00
[L1] Ep 50 | Mean: 119.63 | Prog: -7.18 | Align: -5.44 | Prox: 5.98 | Decel: 0.00 | Tilt: 0.00 | Acti: -1.29 | Arrive: 149.03 | Time: -21.48 | Bound: 0.00

[Det Eval] L1 Ep 50 | Success: 100% -> [Best saved: 100%] [1/1]

=====
[Curriculum] LEVEL UP! 1 -> 2
=====
```

圖二：課程進升紀錄 log

5.2 訓練期不同 level 結果 (每難度 100 回合) (*附錄 B 可見詳細)

難度	成功率 (%)	平均步數	平均獎勵	EffSpd (成功回合)
L1	100%	16.3	198.80	0.1784m/s

難度	成功率 (%)	平均步數	平均獎勵	EffSpd (成功回合)
L2	79%	173.7	166.30	0.0550m/s
L3	97%	76.5	274.70	0.0523m/s
L4	98%	103.3	342.02	0.0522m/s
L5	93%	136.4	376.06	0.0497 m/s
L6	96%	148.3	441.46	0.0457 m/s
L7	82%	213.2	431.91	0.0428 m/s

- L1 因為目標十分接近，所以達成率很高。
- L2 因為目標距離短，所以飛機無法很有效學到訊號代表的意義，因此在測試時成功率略有下降。
- 從 L3 到 L7，平均步數跟獎勵都增加。EffSpd 雖然稍微下降，但幅度不大，代表模型並不是用全程慢飛的保守策略來換取穩定，而是真的在前段維持接近最大速度，到快到了才開始主動減速。

5.3 PPO vs. P 控制器 ($K_p=0.5$ ，最大速度 1 m/s) 對比

指標	P 控制器 (L7)	PPO (L7)
成功率	93%	82%
平均獎勵	449.52	431.91
EffSpd (成功回合)	0.0661 m/s	0.0428 m/s
煞車策略	線性誤差遞減 + 閾值內硬編碼歸零	預測性主動煞車 + 動態平衡
平均步數	141.3	213.2

<pre> ===== Level : 7 Mode: Deterministic RL Success : 82/100 = 82.0% Mean reward : 431.91 Mean steps : 213.2 EffSpd (all) : 0.0483 m/s EffSpd (success): 0.0428 m/s ===== </pre>	<pre> ===== Level : 7 Mode: Baseline Success : 93/100 = 93.0% Mean reward : 449.52 Mean steps : 141.3 EffSpd (all) : 0.0655 m/s EffSpd (success): 0.0661 m/s ===== </pre>
---	---

圖三：PPO vs P 控制器軌跡對比 (L7)

在完全公平的基準下，實驗結果顯示 P 控制器在成功率、平均步數、總獎勵以及有效速度上，**全面優於**訓練完成的 PPO 模型。此現象並非代表強化學習失效，而是凸顯了不同控制典範在特定環境下的特性：

理想動力學下的線性優勢

在 Gazebo 模擬的無風、無隨機動態干擾環境中，無人機的運動學高度線性。P 控制器（速度與距離誤差成正比）能產生一條極度平滑的減速軌跡。相反地，PPO 模型透過 MLP 輸出的非線性指令，在逼近目標時容易產生微小的震盪，導致額外耗時。

「硬編碼歸零」與「動態平衡」的懸停差異

1. P 控制器一旦偵測到無人機進入抵達範圍且速度達標，就強制介入輸出零速度指令。這種硬編碼瞬間鎖死了機身位置，使其能以最高效率吃滿環境給予的懸停品質獎勵。
2. 而 PPO 無法使用這種指令，它必須在連續的動作空間中，不斷微調浮點數輸出以維持動態平衡，這大幅增加了超時或飄出範圍的風險。

複雜獎勵函數導致的保守策略

PPO 的平均步數 (231.2) 顯著較高，反映了 RL 演算法在多目標最佳化下的妥協。為了躲避環境中的減速帶超速懲罰與墜機懲罰，PPO 學會了極度保守的預測性煞車。它寧願提早減速、拉長飛行時間，也不願承擔全速衝刺帶來的墜機風險。而 P 控制器因為不考慮姿態安全，反而能以更直接的軌跡切入目標。

5.4 失敗案例分析

案例一：極端座標傾角墜機

當目標剛好生成在環境邊緣（高度 $z \approx 5\text{m}$ 、 $x/y \approx \pm 9\sim 10\text{m}$ 的角落），無人機為了趕時間會全速衝刺，到了之後為了不飛出邊界就需要瞬間大幅減速，這個瞬間輸出的反向速度指令會讓機身傾角超過 60° 的硬限制，直接被判定墜機。問題的根源在於純速度指令的控制方式無法保證在劇烈減速時的機身姿態安全。

Ep	84/100	Success	Steps: 170	Reward: 529.02	EffSpd: 0.049 m/s	Target: (3.72, -6.73, 3.72)
Ep	85/100	Failure	Steps: 76	Reward: -56.64	EffSpd: 0.169 m/s	Target: (-9.03, -8.22, 4.65)
Ep	86/100	Success	Steps: 184	Reward: 587.66	EffSpd: 0.055 m/s	Target: (-0.03, 9.02, 5.26)

圖四：失敗案例一—傾角墜機軌跡

案例二：低速徘徊／策略崩潰

當目標在 $(-1.79, 0.21, 2.84)$ 這種相對靠近原點的位置，模型輸出的速度指令可能極小，甚至接近零，飛機就在附近非常緩慢地漂移，始終沒有真正進入到達圈，步數跑滿 600 步後超時。

可能的觸發原因：目標距離太短（課程訓練以長距離為主，近距離的位置可能在訓練中極少遇到）、模型還沒完全收斂。

解決辦法：或許可以持續增加訓練量，或是針對極端值進行特化訓練，使飛機模型可以正確辨識位置與輸出速度指令。

Ep	38/100	Success	Steps: 68	Reward: 257.99	EffSpd: 0.047 m/s	Target: (1.70, -1.90, 2.56)
Ep	39/100	Failure	Steps: 600	Reward: -48.29	EffSpd: 0.005 m/s	Target: (-1.79, 0.21, 2.84)
Ep	40/100	Success	Steps: 34	Reward: 201.09	EffSpd: 0.052 m/s	Target: (0.90, -1.17, 1.61)

圖五：失敗案例二—低速徘徊／策略崩潰

附錄 A、Part 1 P 控制器調參心得

調參時固定目標點在 (9, 9, 9) 和 (-9, -9, 9) 之間來回飛，直線距離約 25 公尺，每次只動一個參數。

Kp 調整

- $K_p = 0.1$ ：起步非常慢，而且大概離目標還剩 3 公尺就幾乎動不了了，穩態誤差很嚴重。原因是誤差乘上這麼小的 K_p 之後，產生的速度指令小到系統阻力就把它抵銷掉了。
- $K_p = 1.5$ ：反應快、起步有力，但也只是提早一點點才開始變慢，最後還是停在目標附近一個有點距離的地方，沒辦法完全收斂—純 P 控制器沒有積分項，本來就有穩態誤差這個死穴。

max_speed 調整

- $\text{max_speed} = 10 \text{ m/s}$ ：整體飛行速度明顯提升，到達時間縮短很多，效率改善顯著。
- $\text{max_speed} = 1 \text{ m/s}$ ：飛得很慢，抵達時間大幅拉長，但控制精度一點都沒有因此變好—說明 max_speed 只是限制最高速度，並不會改變控制邏輯本身，穩態誤差的問題還是在。

附錄 B、訓練期不同 level 結果截圖

```

=====
Level : 1 | Mode: Deterministic RL
Success   : 100/100 = 100.0%
Mean reward : 198.80
Mean steps : 16.3
EffSpd (all) : 0.1784 m/s
EffSpd (success): 0.1784 m/s
=====

Level : 2 | Mode: Deterministic RL
Success   : 79/100 = 79.0%
Mean reward : 166.30
Mean steps : 173.7
EffSpd (all) : 0.0444 m/s
EffSpd (success): 0.0550 m/s
=====

Level : 3 | Mode: Deterministic RL
Success   : 97/100 = 97.0%
Mean reward : 274.70
Mean steps : 76.5
EffSpd (all) : 0.0511 m/s
EffSpd (success): 0.0523 m/s
=====

Level : 4 | Mode: Deterministic RL
Success   : 98/100 = 98.0%
Mean reward : 342.02
Mean steps : 103.3
EffSpd (all) : 0.0514 m/s
EffSpd (success): 0.0522 m/s
=====

Level : 5 | Mode: Deterministic RL
Success   : 93/100 = 93.0%
Mean reward : 376.06
Mean steps : 136.4
EffSpd (all) : 0.0500 m/s
EffSpd (success): 0.0497 m/s
=====

Level : 6 | Mode: Deterministic RL
Success   : 96/100 = 96.0%
Mean reward : 441.61
Mean steps : 148.3
EffSpd (all) : 0.0457 m/s
EffSpd (success): 0.0457 m/s
=====

Level : 7 | Mode: Deterministic RL
Success   : 82/100 = 82.0%
Mean reward : 431.91
Mean steps : 213.2
EffSpd (all) : 0.0483 m/s
EffSpd (success): 0.0428 m/s  +  4E 01 54 2A 67 74 (80 9A 00 59)
=====

```

附錄 C、AI 工具輔助

1. 本報告的格式美化、錯字檢查等。
2. 獎勵函數的係數設計，計算在各種情況下是否會過大或過小。
3. Code 的註解美化、增加易讀性。
4. 協助 Code 邏輯錯誤尋找、Bug 修正。
5. 協助環境設置問題。
6. Readme 文檔的格式美化、優化。